

```

import logging
import os
from telegram import Update, ReplyKeyboardMarkup, ReplyKeyboardRemove, KeyboardBu
from telegram.ext import (
    Application,
    CommandHandler,
    MessageHandler,
    ConversationHandler,
    filters,
    ContextTypes,
)

# =====
# ТОКЕН И ID берутся автоматически из Railway
# Ничего здесь менять не нужно!
# =====
TOKEN = os.environ.get("BOT_TOKEN")
ADMIN_CHAT_ID = int(os.environ.get("ADMIN_CHAT_ID"))

# Шаги диалога
CHOOSE_TYPE, ENTER_FLAT, ENTER_NAME, ENTER_PHONE, ENTER_TICKET, ENTER_MESSAGE = r

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    keyboard = [
        ["📄 Обращение"],
        ["💡 Предложение"],
        ["⚠️ Жалоба"],
    ]
    reply_markup = ReplyKeyboardMarkup(keyboard, resize_keyboard=True)
    await update.message.reply_text(
        "👋 Добро пожаловать!\n\n"
        "Это бот для связи жителей с Управляющей компанией.\n\n"
        "Выберите тип обращения:",
        reply_markup=reply_markup,
    )
    return CHOOSE_TYPE

async def choose_type(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text

```

```

if text not in ["📄 Обращение", "💡 Предложение", "⚠️ Жалоба"]:
    await update.message.reply_text("Пожалуйста, выберите пункт из меню.")
    return CHOOSE_TYPE

context.user_data["type"] = text
await update.message.reply_text(
    f"Вы выбрали: {text}\n\n🏠 Введите номер вашей квартиры:",
    reply_markup=ReplyKeyboardRemove(),
)
return ENTER_FLAT

async def enter_flat(update: Update, context: ContextTypes.DEFAULT_TYPE):
    flat = update.message.text.strip()
    if not flat.isdigit():
        await update.message.reply_text("Пожалуйста, введите номер квартиры цифра")
        return ENTER_FLAT

    context.user_data["flat"] = flat
    await update.message.reply_text("👤 Введите ваше ФИО:")
    return ENTER_NAME

async def enter_name(update: Update, context: ContextTypes.DEFAULT_TYPE):
    name = update.message.text.strip()
    if len(name) < 3:
        await update.message.reply_text("Пожалуйста, введите полное ФИО:")
        return ENTER_NAME

    context.user_data["name"] = name

contact_button = KeyboardButton("📞 Отправить мой номер", request_contact=True)
keyboard = ReplyKeyboardMarkup(
    [[contact_button]], resize_keyboard=True, one_time_keyboard=True
)
await update.message.reply_text(
    "📞 Введите ваш номер телефона для обратной связи\n\n"
    "Например: +79001234567\n\n"
    "Или нажмите кнопку ниже, чтобы поделиться номером автоматически:",
    reply_markup=keyboard,
)
return ENTER_PHONE

async def enter_phone(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if update.message.contact:
        phone = update.message.contact.phone_number

```

```

        if not phone.startswith("+"):
            phone = "+" + phone
    else:
        phone = update.message.text.strip()
        digits = (
            phone.replace("+", "")
            .replace("-", "")
            .replace(" ", "")
            .replace("(", "")
            .replace(")", "")
        )
        if not digits.isdigit() or len(digits) < 10:
            await update.message.reply_text(
                "⚠️ Пожалуйста, введите корректный номер телефона.\n"
                "Например: +79001234567"
            )
            return ENTER_PHONE

    context.user_data["phone"] = phone

    keyboard = ReplyKeyboardMarkup(
        [{"➡️ Пропустить"}], resize_keyboard=True, one_time_keyboard=True
    )
    await update.message.reply_text(
        "📁 Если вы уже оставляли заявку в другом сервисе (например, ГИС ЖКХ, сай"
        "введите её номер, чтобы мы могли связать обращения.\n\n"
        "Если заявки нет — нажмите кнопку «Пропустить»:",
        reply_markup=keyboard,
    )
    return ENTER_TICKET


async def enter_ticket(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip()
    context.user_data["ticket"] = None if text == "➡️ Пропустить" else text

    appeal_type = context.user_data["type"]
    await update.message.reply_text(
        f"✍️ Напишите текст вашего обращения ({appeal_type.split()[-1].lower()}):
        reply_markup=ReplyKeyboardRemove(),
    )
    return ENTER_MESSAGE


async def enter_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    message_text = update.message.text.strip()
    user_data = context.user_data

```

```

appeal_type = user_data.get("type", "Обращение")
flat = user_data.get("flat", "-")
name = user_data.get("name", "-")
phone = user_data.get("phone", "-")
ticket = user_data.get("ticket")
username = update.message.from_user.username or "нет username"
user_id = update.message.from_user.id

ticket_line = f"Номер заявки: {ticket}\n" if ticket else "Номер заявки: не ук

admin_message = (
    f" НОВОЕ ОБРАЩЕНИЕ\n"
    f"{'-' * 30}\n"
    f"Тип: {appeal_type}\n"
    f"Квартира: №{flat}\n"
    f"ФИО: {name}\n"
    f"Телефон: {phone}\n"
    f"{ticket_line}"
    f"Telegram: @{username} (ID: {user_id})\n"
    f"{'-' * 30}\n"
    f"Текст:\n{message_text}"
)

try:
    await context.bot.send_message(chat_id=ADMIN_CHAT_ID, text=admin_message)
except Exception as e:
    logger.error(f"Ошибка отправки администратору: {e}")

ticket_confirm = f"Номер заявки: {ticket}\n" if ticket else ""
keyboard = [
    [ Обращение],
    [ Предложение],
    [ Жалоба],
]

reply_markup = ReplyKeyboardMarkup(keyboard, resize_keyboard=True)
await update.message.reply_text(
    f" Ваше обращение принято!\n\n"
    f"Тип: {appeal_type}\n"
    f"Квартира: №{flat}\n"
    f"ФИО: {name}\n"
    f"Телефон: {phone}\n"
    f"{ticket_confirm}"
    "\nУправляющая компания рассмотрит ваше обращение в ближайшее время.\n\n"
    "Хотите отправить ещё одно обращение?",
    reply_markup=reply_markup,
)

```

```

context.user_data.clear()
return CHOOSE_TYPE

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "Диалог отменён. Напишите /start чтобы начать заново.",
        reply_markup=ReplyKeyboardRemove(),
    )
    context.user_data.clear()
    return ConversationHandler.END

def main():
    app = Application.builder().token(TOKEN).build()

    conv_handler = ConversationHandler(
        entry_points=[CommandHandler("start", start)],
        states={
            CHOOSE_TYPE: [MessageHandler(filters.TEXT & ~filters.COMMAND, choose_
            ENTER_FLAT: [MessageHandler(filters.TEXT & ~filters.COMMAND, enter_fl
            ENTER_NAME: [MessageHandler(filters.TEXT & ~filters.COMMAND, enter_na
            ENTER_PHONE: [
                MessageHandler(filters.CONTACT, enter_phone),
                MessageHandler(filters.TEXT & ~filters.COMMAND, enter_phone),
            ],
            ENTER_TICKET: [MessageHandler(filters.TEXT & ~filters.COMMAND, enter_
            ENTER_MESSAGE: [MessageHandler(filters.TEXT & ~filters.COMMAND, enter
        ],
        fallbacks=[CommandHandler("cancel", cancel)],
    )

    app.add_handler(conv_handler)

    print("✅ Бот запущен!")
    app.run_polling()

if __name__ == "__main__":
    main()

```